Python 자료구조와 제어문

학습목표

1) 상황에 맞는 자료구조를 고른다

리스트·튜플·딕셔너리·집합의 차이와 쓰임

2) 조건문으로 흐름을 나눈다

if·elif·else, 들여쓰기, 빈 리스트 처리

3) 반복문으로 여러 값을 처리한다

for·while·break, range, enumerate, zip

4) 함수로 코드를 묶고 오류를 다룬다

인자·반환값·기본값, 람다, try·except

DAY 2의 흐름

노트북 1~3절

튜플 · 딕셔너리 · 집합

언패킹, get·update·del, 집합 연산

노트북 4절

조건문

if·elif·else, 들여쓰기, 참·거짓 판정

노트북 5절

반복문

for·while·break, range, enumerate·zip

노트북 6절

함수

인자·반환·기본값, 지역변수, 람다

노트북 7~8절

오류와 종합 실습

try·except, 매출 데이터 요약

튜플 · 딕셔너리 · 집합

튜플

튜플

- 튜플은 여러 값을 순서대로 저장하며, 생성 후 항목을 변경할 수 없는 자료형이다
- 보통 소괄호로 표현하며, 쉼표로 구분하여 생성한다

  예: spec = (60, 90)
- 바뀌면 안 되는 값(규격, 좌표)이나 함수가 여러 값을 돌려줄 때 쓴다
- 언패킹: low, high = spec 처럼 값을 변수 여러 개에 한 번에 나누어 담는다
- 값의 개수와 변수의 개수가 같아야 한다

```
spec = (60, 90)
```

튜플 (60, 90)
값 2개가 한 묶음

```
low, high = spec (언패킹)
```

↓

low	high
60	90

값의 개수와 변수의 개수가 같아야 한다

언패킹은 여러 값이 묶여 있는 것을 풀어서 각 변수에 나누어 담는 것이다

리스트는 바꿀 수 있고, 튜플은 바꿀 수 없다. 바뀌면 안 되는 값은 튜플로 둔다

튜플과 시퀀스(한국어): https://docs.python.org/ko/3.13/tutorial/datastructures.html#tuples-and-sequences

튜플 사용 예

```
# 튜플: 바뀌면 안 되는 범위 값을 묶는다
spec = (60, 90)
low, high = spec            # 언패킹: 값 2개를 변수 2개에 나누어 담는다
print(low, high)            # 60 90

# 함수가 두 값을 돌려줄 때도 튜플로 묶는다
def min_max(values):
    return min(values), max(values)
lo, hi = min_max([72.1, 73.5, 74.2])
print(lo, hi)               # 72.1 74.2
```

주의

- spec[0] = 1 처럼 튜플의 값을 바꾸려 하면 TypeError 가 발생한다
- 값을 바꿔야 하면 처음부터 리스트로 만든다
- 언패킹할 때 변수의 개수가 값의 개수와 다르면 ValueError 가 발생한다

리스트와 튜플 비교

구분	리스트	튜플
만들기	[60, 90]	(60, 90)
값 바꾸기	sales[0] = 100 가능	불가능 (TypeError)
추가 · 삭제	append(), remove()	불가능
쓰는 상황	값이 늘거나 바뀌는 목록	좌표, 허용 범위, 함수의 여러 반환값
쓸 수 있는 메서드	많다	count(), index() 두 개뿐

값 여러 개를 쉼표로 적으면 Python 은 자동으로 튜플로 묶는다: spec = 60, 90

딕셔너리

딕셔너리

- 키(key)와 값(value)의 쌍을 저장한다
- 위치 번호가 아니라 키로 값을 찾는다
- 키는 중복될 수 없다
- 중괄호 { 키: 값 } 으로 만든다

```
# 딕셔너리: 키로 값을 찾는다
region_sales = {"서울": 350000, "부산": 210000}
print(region_sales["서울"])           # 350000

region_sales["대구"] = 180000        # 새 키 추가 (있는 키면 수정)
print(region_sales.get("제주", 0))   # 0: 없는 키는 기본값

for region, amount in region_sales.items():   # (키, 값) 쌍을 하나씩
    print(region, amount)
```

주의

- 없는 키를 [ ] 로 조회하면 KeyError 가 발생한다
- 키가 있는지 확신이 없으면 get(키, 기본값) 을 쓴다
- 같은 키에 다시 대입하면 값이 덮어써진다 (키는 중복될 수 없다)

딕셔너리(한국어): https://docs.python.org/ko/3.13/tutorial/datastructures.html#dictionaries

딕셔너리 주요 문법

d = {"서울": 500, "부산": 300}

문법	의미	예시
`{키: 값}`	딕셔너리 만들기	`{"서울": 500}`
`d[키]`	값 조회	`d["서울"] → 500`
`d[키] = 값`	추가 또는 수정	`d["대구"] = 180`
`d.get(키, 기본값)`	없는 키도 안전하게 조회	`d.get("제주", 0) → 0`
`d.keys()`	키 목록	`dict_keys(['서울', '부산'])`
`d.values()`	값 목록	`dict_values([500, 300])`
`d.items()`	(키, 값) 쌍 목록	`for k, v in d.items():`

딕셔너리(한국어): https://docs.python.org/ko/3.13/tutorial/datastructures.html#dictionaries
딕셔너리 뷰 객체(한국어): https://docs.python.org/ko/3.13/library/stdtypes.html#dictionary-view-objects

딕셔너리 안전하게 다루기: get · update · del

동작	코드	결과
있는 키 조회	`sales["서울"]`	350000
없는 키 조회 (대괄호)	`sales["대전"]`	KeyError 오류
없는 키 조회 (get)	`sales.get("대전", 0)`	0 (기본값을 돌려준다)
여러 값 한 번에 추가·수정	`sales.update({"대전": 210000, "부산": 300000})`	대전은 추가, 부산은 수정된다
키 삭제	`del sales["부산"]`	부산 항목이 사라진다

totals[키] = totals.get(키, 0) + 값

집합

집합

- 중복이 없는 값의 묶음이다. 순서가 없어서 인덱스로 꺼낼 수 없다
- set(리스트) 로 만들면 중복이 하나로 합쳐진다
- 고유한 종류의 수, 두 목록의 공통 항목 · 차이를 구할 때 쓴다
- 출력 순서가 화면과 다를 수 있다
- 순서가 필요하면 sorted(set(x)) 로 정렬된 리스트를 만든다

문법	의미	예시
`set(리스트)`	집합 만들기 (중복 제거)	`set([1, 1, 2]) → {1, 2}`
`len(집합)`	고유한 값의 개수	`len({1, 2}) → 2`
`a & b`	교집합: 둘 다에 있는 값	`{1, 2} & {2, 3} → {2}`
`a | b`	합집합: 하나라도 있는 값	`{1, 2} | {2, 3} → {1, 2, 3}`
`a - b`	차집합: a 에만 있는 값	`{1, 2} - {2, 3} → {1}`

집합(한국어): https://docs.python.org/ko/3.13/tutorial/datastructures.html#sets

집합 · 중복 제거

machines
기록 5건

M01	M02	M01	M03	M02
		중복		중복

↓ `set(machines)` 중복이 하나로 합쳐진다

unique
설비 3종류

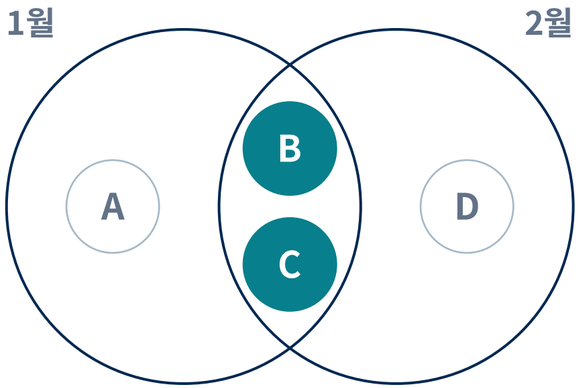
M01	M02	M03

```
machines = ["M01", "M02", "M01", "M03", "M02"]
unique = set(machines)
print(len(machines))    # 5: 기록 수
print(len(unique))      # 3: 설비 종류 수
print(sorted(unique))   # ['M01', 'M02', 'M03']  순서가 필요하면 정렬
```

집합 연산: 교집합 · 합집합 · 차집합

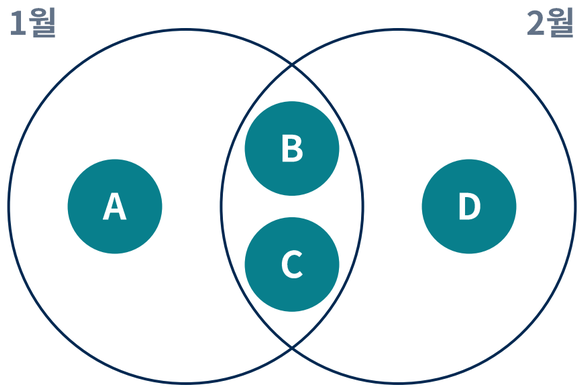
1월 주문 고객 {A, B, C} 와 2월 주문 고객 {B, C, D} 가 있을 때

& 교집합



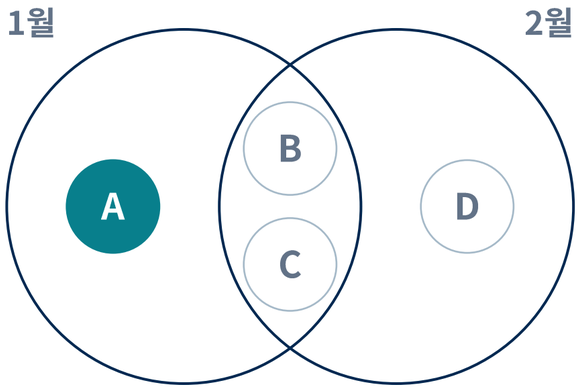
{B, C} 두 달 모두 주문

| 합집합



{A, B, C, D} 한 번이라도

- 차집합



{A} 1월에만 주문

```
january = {"A", "B", "C"}
february = {"B", "C", "D"}
print(january & february)   # {'B', 'C'}
print(january | february)   # {'A', 'B', 'C', 'D'}
print(january - february)   # {'A'}      (출력 순서는 다를 수 있다)
```

조건문

조건문

조건문

- 조건의 참·거짓에 따라 실행할 코드를 선택한다
- if 조건: / elif 조건: / else: 순서로 쓴다
- 조건 뒤에는 콜론(:)을 쓰고, 다음 줄은 들여쓴다 (공백 4칸 권장)
- 위에서부터 차례로 검사하고, 처음 참인 블록 하나만 실행한다
- elif 와 else 는 필요 없으면 생략할 수 있다

기본 형태

```
if 조건1:
    조건1 이 참일 때 실행
elif 조건2:
    조건1 은 거짓, 조건2 가 참일 때
else:
    모두 거짓일 때
```

- 비교 연산: ==, !=, >, >=, <, <=
- 여러 조건은 and, or 로 묶는다
- not 은 참·거짓을 뒤집는다

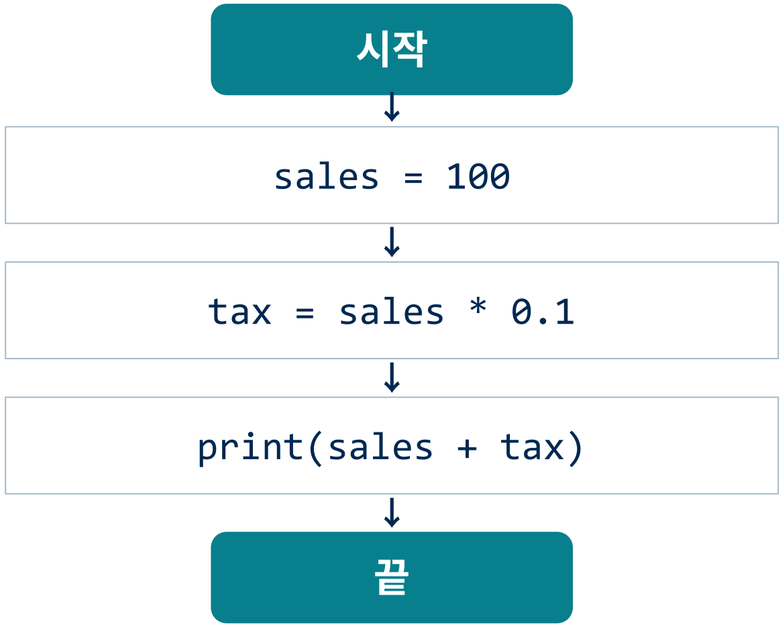
콜론(:)과 들여쓰기가 조건문의 범위를 정한다. 같은 블록은 같은 칸만큼 들여쓴다

if 문(한국어): https://docs.python.org/ko/3.13/tutorial/controlflow.html#if-statements
코딩 스타일(한국어): https://docs.python.org/ko/3.13/tutorial/controlflow.html#intermezzo-coding-style

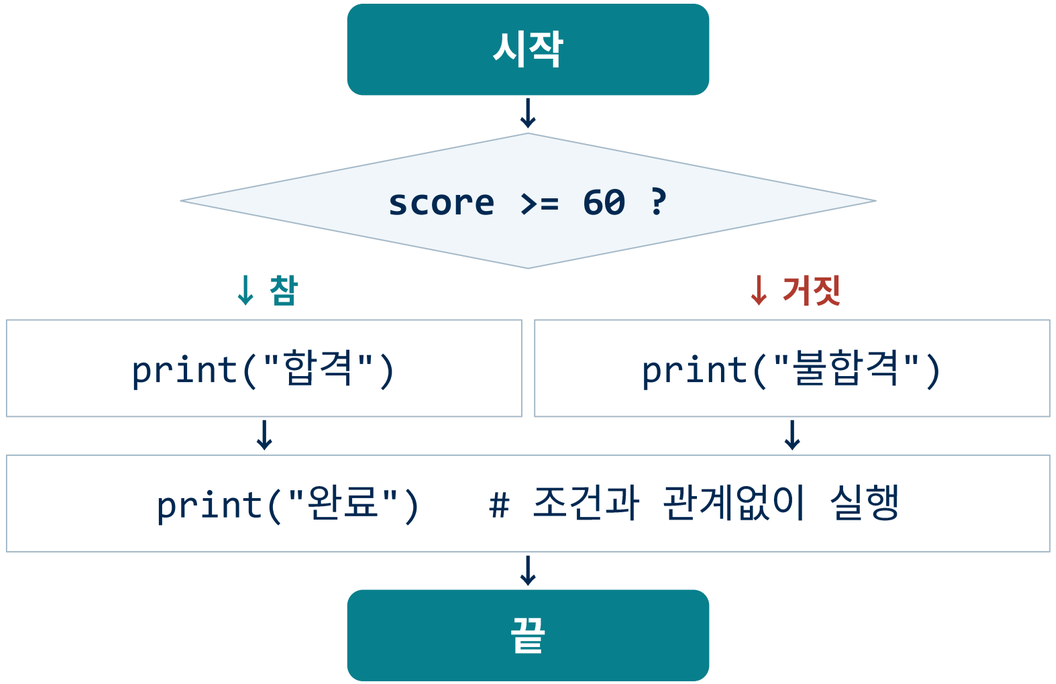
순차문과 조건문의 흐름

순차문은 위에서 아래로 차례대로 실행하고, 조건문은 판단 결과에 따라 실행할 문장이 달라진다.

순차문: 차례대로 실행



조건문: 조건에 따라 선택 실행



조건문 사용 예

```
# 불량률에 따라 등급을 정한다
defect_rate = 3.8
if defect_rate <= 2:
    grade = "양호"          # 3.8 <= 2 → 거짓, 건너뛴다
elif defect_rate <= 5:
    grade = "주의"          # 3.8 <= 5 → 참, 여기만 실행
else:
    grade = "점검 필요"
print(grade)                 # 주의
```

들여쓰기와 실행 범위

1) 두 문장 모두 조건문 안

```
sales = 50
if sales >= 100:
    print("기준 이상")
    print("확인 완료")

# 조건이 거짓이므로 출력 없음
```

두 print 모두 들여쓰기가 되어 있어 조건이 참일 때만 실행된다

2) 마지막 문장은 조건문 밖

```
sales = 50
if sales >= 100:
    print("기준 이상")
print("확인 완료")

# 확인 완료
```

마지막 print 는 들여쓰기가 없어 조건과 관계없이 항상 실행된다

참·거짓 판정 (Truth Value Testing)

거짓으로 취급되는 값

`None`, `False`
`0`, `0.0` (숫자 0)
`""` (빈 문자열)
`[]` `()` (빈 리스트, 빈 튜플)
`{}` `set()` (빈 딕셔너리, 빈 집합)

그 외 모든 값은 참

빈 리스트 먼저 확인하기

```
sales = []

if sales:                    # 비어 있으면 거짓
    print(sum(sales) / len(sales))
else:
    print("데이터 없음")  # 0 나누기 예방
```

출력: 데이터 없음

평균을 내기 전에 `if sales:` 또는 `if len(sales) > 0:` 으로 빈 목록을 거른다

Truth Value Testing(한국어): https://docs.python.org/ko/3.13/library/stdtypes.html#truth-value-testing

반복문

반복문을 사용하는 이유

반복문 없이

```
sales = [120000, 150000, 90000]
print(sales[0])        # 120000
print(sales[1])        # 150000
print(sales[2])        # 90000
# 1,000건이면 1,000줄…
# 값이 늘면 코드를 다시 고쳐야 한다
```

반복문으로

```
sales = [120000, 150000, 90000]
for value in sales:    # 값을 하나씩 꺼내서
    print(value)       # 출력
# 120000
# 150000
# 90000     ← 3건이든 1,000건이든 코드는 2줄
```

각 값에 계산을 해야 할 때도 같다

```
print(sales[0] * 1.1)  # 10% 인상가
print(sales[1] * 1.1)
print(sales[2] * 1.1)
```

```
for value in sales:
    print(value * 1.1)   # 값마다 같은 계산
```

같은 일을 여러 값에 되풀이할 때 반복문을 쓴다

for 문(한국어): https://docs.python.org/ko/3.13/tutorial/controlflow.html#for-statements

반복문

반복문

- 같은 작업을 여러 값에 반복해서 적용한다
- for 변수 in 리스트: 형태로 쓴다
- 콜론(:)과 들여쓰기로 반복할 범위를 정한다
- for 안에 if 를 넣어 조건에 맞는 값만 처리할 수 있다

기본 형태

```
sales = [120000, 150000, 90000]
for value in sales:
    print(value)      # 값마다 한 번씩
```

- value 는 리스트에서 꺼낸 값을 담는 변수
- 리스트 자리에는 문자열·튜플·range() 처럼 값을 하나씩 꺼낼 수 있는 것이면 무엇이든 올 수 있다
- 공식 문서는 이것을 이터러블(iterable)이라 부른다: "멤버들을 한 번에 하나씩 돌려줄 수 있는 객체"

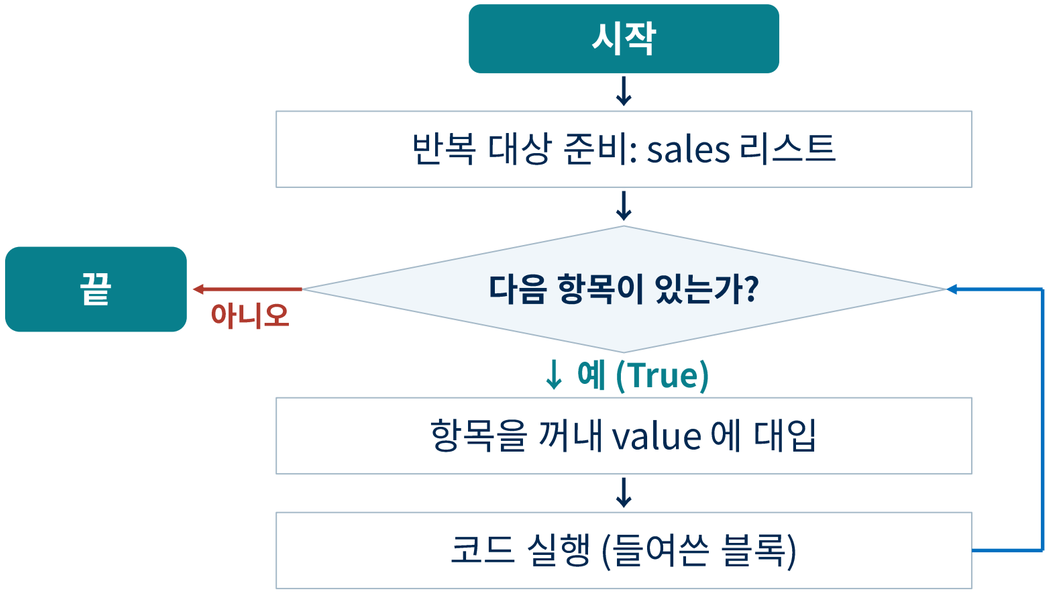
함수	역할	예시
`range(n)`	0 부터 n-1 까지 숫자를 만든다 (n 번 반복)	`for i in range(3):  → 0, 1, 2`
`enumerate(리스트)`	순번과 값을 함께 꺼낸다	`for i, v in enumerate(sales):`
`zip(리스트1, 리스트2)`	두 리스트의 같은 위치 값을 짝지어 꺼낸다	`for r, a in zip(regions, sales):`

for 문(한국어): https://docs.python.org/ko/3.13/tutorial/controlflow.html#for-statements · 용어집 이터러블(한국어): https://docs.python.org/ko/3.13/glossary.html#term-iterable
range() 함수(한국어): https://docs.python.org/ko/3.13/tutorial/controlflow.html#the-range-function

for 와 while 의 흐름

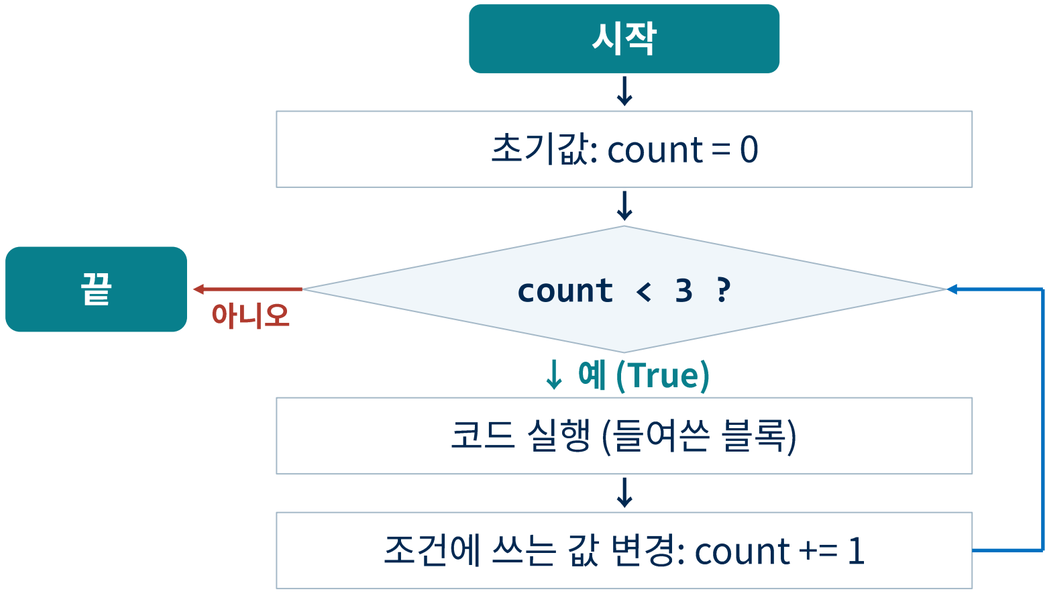
for 는 리스트의 다음 항목이 있는 동안 반복하고, while 은 조건이 참인 동안 반복한다.

for: 항목을 하나씩 꺼내며 반복




```
sales = [120000, 150000, 90000]   # 반복 대상 준비
for value in sales:               # 다음 항목을 꺼내 대입
    print(value)                  # 코드 실행
```

while: 조건이 참인 동안 반복




```
count = 0                         # 초기값
while count < 3:                  # 조건 판단
    print(count)                  # 코드 실행
    count += 1                    # 조건에 쓰는 값 변경
```

컴프리헨션(comprehension)

컴프리헨션(Comprehension)은 반복 가능한 데이터의 요소에 표현식과 선택적인 조건을 적용하여 리스트·집합·딕셔너리 등을 간결하게 생성하는 문법이다.

values = [50, 100, 150, 100] 일 때

괄호	코드	결과
[ ] 리스트	`[v for v in values if v >= 100]`	[100, 150, 100]
{ } 집합	`{v for v in values if v >= 100}`	{100, 150} 중복 사라짐
{키: 값} 딕셔너리	`{v: v*2 for v in values if v >= 100}`	{100: 200, 150: 300}

[v * 2 for v in values] 처럼 조건이 없어도 컴프리헨션

for 문과 리스트 컴프리헨션

1) for 문

```
values = [50, 100, 150]
over = []
for v in values:
    if v >= 100:
        over.append(v)
print(over)   # [100, 150]
```

2) 리스트 컴프리헨션

```
values = [50, 100, 150]
over = [v for v in values if v >= 100]
print(over)   # [100, 150]
```

문법	설명
v	결과 리스트에 저장할 값
for v in values	values의 요소를 하나씩 가져옴
if v >= 100	100 이상인 값만 선택
[ ]	결과를 새로운 리스트로 생성

리스트 컴프리헨션(한국어): https://docs.python.org/ko/3.13/tutorial/datastructures.html#list-comprehensions

range(): 정해진 횟수만큼 반복

형태	만들어지는 숫자	뜻
`range(5)`	`0, 1, 2, 3, 4`	0 부터 5 직전까지 (5 번 반복)
`range(1, 6)`	`1, 2, 3, 4, 5`	1 부터 6 직전까지
`range(0, 60, 15)`	`0, 15, 30, 45`	0 부터 15 간격으로
`for i in range(3):`	`i = 0, 1, 2`	3 번 반복

리스트 슬라이싱과 같은 규칙: 시작은 포함, 끝은 포함하지 않는다

range() 함수(한국어): https://docs.python.org/ko/3.13/tutorial/controlflow.html#the-range-function · range(한국어): https://docs.python.org/ko/3.13/library/stdtypes.html#range

while · break · continue

`while 조건:`	`break`	`continue`
조건이 참인 동안 반복	반복을 즉시 끝낸다	이번 값만 건너뛰고 다음 값으로

while: 조건이 참인 동안

```
count = 0
while count < 3:
    print(count)   # 0, 1, 2
    count += 1     # 조건을 바꾸는 줄
```

for 안에서 break · continue

```
total = 0
for value in sales:
    if value < 0:
        continue        # 음수는 건너뛴다
    if value > 1000000:
        break           # 100만 초과면 멈춘다
    total += value
```

break 와 continue 문(한국어): https://docs.python.org/ko/3.13/tutorial/controlflow.html#break-and-continue-statements
for 문(한국어): https://docs.python.org/ko/3.13/tutorial/controlflow.html#for-statements

break · continue 값 하나씩 따라가기

```
total = 0
for value in sales:
    if value < 0:
        continue
    if value > 1000000:
        break
    total += value
```

`sales = [120000, -5000, 300000, 2000000, 80000]` 을 앞 장의 for 문으로 돌리면

순서	value	확인	동작	total
1	120000	음수 아님, 100만 이하	**더한다**	120000
2	-5000	value < 0 → 참	**continue: 건너뛴다**	120000
3	300000	음수 아님, 100만 이하	**더한다**	420000
4	2000000	value > 1000000 → 참	**break: 반복 끝**	420000
5	80000	확인하지 않는다	**실행되지 않는다**	420000

continue 는 이번 값만 건너뛰고 다음 값으로, break 는 남은 값이 있어도 반복을 끝낸다. 최종 total = 420000

break 와 continue 문(한국어): https://docs.python.org/ko/3.13/tutorial/controlflow.html#break-and-continue-statements

enumerate() 와 zip()

enumerate: 순서가 있는 자료형(리스트, 튜플 등)을 입력받아, 각 요소의 '인덱스(번호)'와 '값'을 동시에 차례대로 반환해 주는 함수

```
regions = ["서울", "부산", "대전"]

for i, region in enumerate(regions, 1):
    print(i, region)
# 1 서울
# 2 부산
# 3 대전
```

enumerate(regions, 1) 의 1 은 순번의 시작값이다. 생략하면 0 부터 센다

zip: 여러 개의 순서가 있는 자료형(리스트, 튜플 등)을 입력받아, 같은 인덱스에 위치한 요소들을 하나로 묶어서 차례대로 반환해 주는 함수

```
regions = ["서울", "부산", "대전"]
sales = [350000, 280000, 210000]
for region, value in zip(regions, sales):
    print(region, value)
# 서울 350000
# 부산 280000
# 대전 210000
```

결과가 (값1, 값2) 튜플이라 for 에서 변수 두 개(region, value)로 받는다

함수	핵심 목적	결과
`enumerate()`	하나의 리스트에서 순번(번호)이 필요할 때	(순번, 값)
`zip()`	여러 리스트를 같은 위치끼리 묶어서 볼 때	(리스트1의 값, 리스트2의 값, …)

enumerate(한국어): https://docs.python.org/ko/3.13/library/functions.html#enumerate · zip(한국어): https://docs.python.org/ko/3.13/library/functions.html#zip

함수

함수

함수

- 함수는 여러 번 쓰는 계산에 이름을 붙여 재사용하는 것이다
- def 이름(매개변수): 로 만들고, return 으로 결과를 돌려준다
- 매개변수(parameter)는 함수가 입력받는 값의 자리이고, 호출할 때 넣는 실제 값을 인자(argument)라고 한다
- 함수는 정의만으로는 실행되지 않고, 이름(인자) 로 호출해야 실행된다
- print 는 화면에 보여주기만 하고, return 은 값을 돌려주어 다음 계산에 쓸 수 있게 한다

sales = [120000, 150000, 90000, 285000] 일 때

```
# 정의부
def average(values):
    return sum(values) / len(values)
```

호출부

```
avg = average(sales)
```

1. 인자 전달
sales → values

2. 반환값 대입
161250.0 → avg

호출하면 인자가 매개변수에 전달되고,
return 값이 호출한 자리로 돌아와 변수에 대입된다

함수 정의하기(한국어): https://docs.python.org/ko/3.13/tutorial/controlflow.html#defining-functions

함수

엑셀의 AVERAGE 는 이미 만들어진 함수이고, Python 에서는 def 로 같은 계산을 직접 만든다.

fx **=AVERAGE(B2:B5)**

	A	B
1	**주문**	**매출**
2	1	120000
3	2	150000
4	3	90000
5	4	285000
6	**평균**	**161250**

B2:B5 를 넣으면 B6 에 결과

```
def average(values):
    return sum(values) / len(values)

sales = [120000, 150000, 90000, 285000]
avg = average(sales)   # 161250.0
```

	엑셀	Python
함수 이름	AVERAGE	average
입력 (인자)	B2:B5 범위	sales 리스트
결과 (반환값)	B6 칸에 표시	avg 변수에 대입

함수의 4가지 형태

입력(인자)이 있는지, 결과를 return 으로 돌려주는지에 따라 네 가지로 나뉜다. 데이터 분석에서는 4) 를 가장 많이 쓴다

1) 입력 없음 · 반환 없음

```
def hello():
    print("안녕하세요")

hello()         # 안녕하세요
```

인자도 없고 돌려주는 값도 없다.

2) 입력 있음 · 반환 없음

```
def show_total(a, b):
    print(a + b)

show_total(2, 3)   # 5
```

a, b 에 2, 3 이 들어가 더한 값을 화면에만 보여 준다

3) 입력 없음 · 반환 있음

```
def get_rate():
    return 0.1

price = 10000 * get_rate()   # 1000.0
```

인자 없이 정해진 값을 돌려준다.

4) 입력 있음 · 반환 있음

```
def plus(a, b):
    return a + b

total = plus(2, 3)
print(total * 10)   # 50
```

값을 받아 계산하고 돌려준다. return 한 값은 변수에 담아 다음 계산에 쓴다

print 는 화면에 보여 주고 끝나지만, return 은 값을 호출한 자리로 돌려준다. 결과를 저장하거나 다시 계산하려면 반환이 있는 3)·4) 모양으로 만든다

return 이 없는 함수

```
def f1():
    print("안녕")       # return 없이 끝에 도달 → None 반환

def f2():
    return              # 값 없이 return 만 사용 → None 반환

def f3():
    return None         # 명시적으로 None 반환

result = f1()           # 화면에 안녕 이 출력된다
print(result)           # None
```

- print 만 있는 함수의 결과를 변수에 담으면 None 이 들어간다
- 계산 결과를 다음에 쓰려면 반드시 return 으로 돌려준다

함수의 기본값

매개변수(parameter)에 기본값이 있으면 호출할 때 그 인자를 생략할 수 있다. 다른 값을 주면 그 값으로 실행한다.

```
def is_high(value, threshold=100):   # threshold 는 기본값이 100 인 매개변수
    return value >= threshold

print(is_high(120))          # True:  기본 기준 100 과 비교
print(is_high(120, 150))     # False: 기준을 150 으로 바꿔 비교
```

- value 와 threshold 는 함수 안에서 사용하는 매개변수명이다
- 기본값이 있는 매개변수는 기본값이 없는 매개변수 뒤에 둔다

기본 인자 값(한국어): https://docs.python.org/ko/3.13/tutorial/controlflow.html#default-argument-values

결과 반환과 언패킹

return 뒤에 값을 쉼표로 나열하면 파이썬이 그 값들을 튜플 하나로 묶어서 돌려준다(튜플 패킹).
받는 쪽에서는 언패킹으로 나누어 담는다.

```
def get_summary(values):
    count = len(values)
    total = sum(values)
    return count, total          # 두 값을 쉼표로 돌려준다 → 튜플 (count, total)

n, amount = get_summary([100, 200, 300])    # 언패킹: 두 변수에 나누어 담는다
print(n, amount)                 # 3 600
```

1) **return count, total 은 (3, 600) 을 반환한다**
2) **n, amount = ... 는 두 값을 차례로 나누어 받는다**

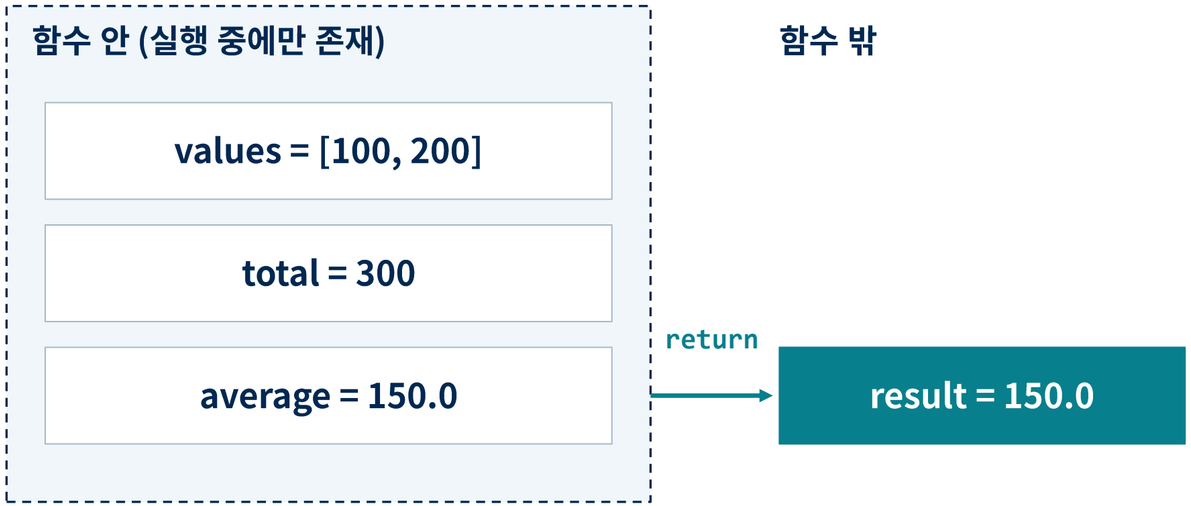
함수 정의하기(한국어): https://docs.python.org/ko/3.13/tutorial/controlflow.html#defining-functions
튜플과 시퀀스(한국어): https://docs.python.org/ko/3.13/tutorial/datastructures.html#tuples-and-sequences

지역변수

함수 내부에서 만든 지역변수는 일반적으로 함수 외부에서 이름으로 직접 접근할 수 없다.

```
def calc_average(values):
    total = sum(values)       # 지역 변수
    average = total / len(values)
    return average

result = calc_average([100, 200])
print(result)   # 150.0
```



`print(total)` → NameError
`print(average)` → NameError

람다(Lambda)

이름 없이 한 줄로 간결하게 만들어서 쓰는 '익명 함수(Anonymous function)'
주로 간단한 계산이나 다른 함수에 짧은 함수를 전달할 때 쓴다.

기본 문법 `lambda 매개변수: 표현식` **함수 내부에서 return 키워드를 명시하지 않아도, 표현식의 계산 결과가 자동으로 반환**

```
orders = [("서울", 300), ("부산", 500), ("대전", 200)]   # (지역, 매출)
```

def 로 쓰면

```
def get_sales(row):
    return row[1]

sorted(orders, key=get_sales)
```

람다로 쓰면

```
sorted(orders, key=lambda row: row[1])

# lambda 입력: 돌려줄 값
# orders 의 항목 하나가 row 에 입력되면
# row[1](매출)을 기준으로 정렬한다
```

두 코드의 결과 (매출기준 오름차순): `[('대전', 200), ('서울', 300), ('부산', 500)]`

람다 표현식(한국어): https://docs.python.org/ko/3.13/tutorial/controlflow.html#lambda-expressions · sorted(한국어): https://docs.python.org/ko/3.13/library/functions.html#sorted

오류 처리

오류 확인

KeyError: 딕셔너리에 없는 키

```
stock = {"볼트": 120, "너트": 80}
stock["볼트1"]
```

```
KeyError: '볼트1'
```

해결

```
stock.get("볼트1", 0)   # 0
```

키 철자를 확인하고, 없을 수 있으면 get(키, 기본값)

ValueError: 값의 내용이 맞지 않음

```
quantity = int("12개")
```

```
ValueError: invalid literal for int()
with base 10: '12개'
```

해결

```
quantity = int("12")    # 12
```

숫자만 남기거나, 다음 장의 try · except 로 처리한다

내장 예외(한국어): https://docs.python.org/ko/3.13/library/exceptions.html
KeyError: https://docs.python.org/ko/3.13/library/exceptions.html#KeyError
ValueError: https://docs.python.org/ko/3.13/library/exceptions.html#ValueError

try · except: 오류가 나도 멈추지 않게 처리

try 블록에서 지정한 종류의 예외가 발생하면 해당 except 블록에서 처리하여 프로그램의 다음 작업을 계속할 수 있다.
처리하지 않은 예외는 실행을 중단시킬 수 있다.

```
for value in ["12", "12개", "7"]:                    # 값을 하나씩 꺼낸다
    try:
        quantity = int(value)                        # 숫자로 바꿔 본다. "12개"는 여기서 실패
        print(quantity)                              # 성공한 값만 이 줄까지 온다
    except ValueError:                               # 위에서 ValueError 가 나면 여기로 건너온다
        print(value, "→ 숫자로 바꿀 수 없다")

print("확인 끝")                                     # 오류가 났어도 멈추지 않고 여기까지 온다
```

출력

```
12                                  # "12" → try 성공
12개 → 숫자로 바꿀 수 없다          # "12개" → except
7                                   # "7" → try 성공
확인 끝                             # 반복이 끝난 뒤 실행
```

try 가 없었다면 "12개"에서 빨간 오류가 뜨고, 7 과 '확인 끝'은 출력되지 않는다

except 뒤에는 잡을 오류의 이름을 적는다

```
except ValueError:                  # 이 오류 하나만
except (ValueError, KeyError):      # 여러 개를 괄호로 묶어서
except Exception:                   # 거의 모든 오류를 한 번에
```

예외 처리하기(한국어): https://docs.python.org/ko/3.13/tutorial/errors.html#handling-exceptions